

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_04_)

Use of Venus Simulator for Register Level Execution of RISC-V Arithmetic and Memory Instructions

Student Name: ___Areeb Fatima___ Roll No: ___24I-6543___ Section: _B_

Date performed: ___9th September___, 2026

MARKS AWARDED: ___ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Task 1:

Description:

In this task, we were asked to perform the arithmetic operations in RISC-V assembly language, for that we generate the blueprint to initialize the data with the values of A and B. After that, in main we load the words and perform the operations by the common syntax of RISC-V as opcode ds1,sr1,sr2 and then we wrote the decimal equivalent to the machine code hexadecimal values as required in part 5 of this task.

Code:

.data

A: .word 5

B: .word 4

.text

.globl main

main:

lw t0, A #5

lw t1, B #4

add s0, t0, t1 #C=A+B

sub s1, t0, t1 #D=A-B

mul s2, t0, t1 #E=A*B

mul s3, t0, t0 #F=A^2

li a7, 93

li a0, 0

ecall

0x10

0x00628433

add x8 x5 x6

add s0, t0, t1 #C=A+B

For Add Operation:

The decimal equivalent is: 6456371

```
0x14      0x406284B3      sub x9 x5 x6      sub s1, t0, t1 #D=A-B
```

For Subtract Operation:

The decimal equivalent is: 1080198323

```
0x18      0x02628933      mul x18 x5 x6      mul s2, t0, t1 #E=A*B
```

For Multiplication Operation:

The decimal equivalent is: 40012083

```
0x1c      0x025289B3      mul x19 x5 x5      mul s3, t0, t0 #F=A^2
```

For Power Operation:

The decimal equivalent is: 38963635

Output:

Run Step Prev Reset Dump Trace					
0x0	0x10000297	auipc x5 65536	lw t0, A #5	s2	20
0x4	0x0002A283	lw x5 0(x5)	lw t0, A #5	(x18)	
0x8	0x10000317	auipc x6 65536	lw t1, B #4	s3	25
0xc	0xFFC32303	lw x6 -4(x6)	lw t1, B #4	(x19)	
0x10	0x00628433	add x8 x5 x6	add s0, t0, t1 #C=A+B	s4	0
0x14	0x406284B3	sub x9 x5 x6	sub s1, t0, t1 #D=A-B	(x20)	
0x18	0x02628933	mul x18 x5 x6	mul s2, t0, t1 #E=A*B	s5	0
0x1c	0x025289B3	mul x19 x5 x5	mul s3, t0, t0 #F=A^2	(x21)	
0x20	0x05D00893	addi x17 x0 93	li a7, 93	s6	0
				(x22)	

Task 2:

Description:

In this task, we were asked to write the equation in assembly language. For that, we use the values from first task and as RISC-V operates on two operands so, we wrote the code by doing addition on two operands and subtraction simultaneously. After that multiplying the equation and adding the whole to that we got the desired output for the equation in saved register as s7 stated in equation as m.

Code:

#task 2

```
sub s4, t0, s0 #a-c
```

```
add s5, s1, s2 #d+e
```

```
mul s6, s5, s3 #(d+e)*f
```

```
add s7, s4, s6 #m
```

This task is embedded in task 01, for verifying the outputs

```

1 .data
2     A: .word 5
3     B: .word 4
4 .text
5 .globl main
6 main:
7     lw t0, A #5
8     lw t1, B #4
9     add s0, t0, t1 #C=A+B
10    sub s1, t0, t1 #D=A-B
11    mul s2, t0, t1 #E=A*B
12    mul s3, t0, t0 #F=A^2
13    #task 2
14    sub s4, t0, s0 #a-c
15    add s5, s1, s2 #d+e
16    mul s6, s5, s3 #(d+e)*f
17    add s7, s4, s6 #m
18    li a7, 93
19    li a0, 0
20    ecall
21

```

Output:

Run Step Prev Reset Dump Trace				(x23)
0x10	0x00628433	add x8 x5 x6	add s0, t0, t1 #C=A+B	s2 (x18) 20
0x14	0x406284B3	sub x9 x5 x6	sub s1, t0, t1 #D=A-B	s3 (x19) 25
0x18	0x02628933	mul x18 x5 x6	mul s2, t0, t1 #E=A*B	s4 (x20) -4
0x1c	0x025289B3	mul x19 x5 x5	mul s3, t0, t0 #F=A^2	s5 (x21) 21
0x20	0x40828A33	sub x20 x5 x8	sub s4, t0, s0 #a-c	s6 (x22) 525
0x24	0x01248AB3	add x21 x9 x18	add s5, s1, s2 #d+e	s7 (x23) 521
0x28	0x033A8B33	mul x22 x21 x19	mul s6, s5, s3 #(d+e)*f	
0x2c	0x016A0BB3	add x23 x20 x22	add s7, s4, s6 #m	
0x30	0x05D00893	addi x17 x0 93	li a7, 93	

Task 3:

Description:

In this task, we were given the two hexadecimal numbers, and we must perform the AND, OR and XOR operations on them. So, for that we simply wrote the two numbers in data and then load the words from memory to register and then wrote the commands for and, or and xor and performed the operation on numbers and store in temporary register. In simulator, we verify the results for the respected operations.

Code:

```
.data

var1: .word 0xffff

var2: .word 0xf0f

.text

.globl main

main: lw t0, var1

      lw t1, var2

      and t2, t0, t1

      or t3, t0, t1

      xor t4, t0, t1

      li a7, 93

      li a0, 0

      ecall
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x000012B7	lui x5 1	main: li t0, 0xffff
0x4	0xFFFF28293	addi x5 x5 -1	main: li t0, 0xffff
0x8	0x00001337	lui x6 1	li t1, 0xf0f
0xc	0xF0F30313	addi x6 x6 -241	li t1, 0xf0f
0x10	0x0062F3B3	and x7 x5 x6	and t2, t0, t1
0x14	0x0062EE33	or x28 x5 x6	or t3, t0, t1
0x18	0x0062CEB3	xor x29 x5 x6	xor t4, t0, t1
0x1c	0x05D00893	addi x17 x0 93	li a7, 93

Task 4:

Description:

In this task, we were given the code in assembly language in which we have to understand and run the code, so firstly I make array in data section and assign 3 values as 20,30,40 and then I make the main section in which I load the array and the size and then jump to arraysum section in which we basically calculate the sum of array by loop and left shift by 2 for 4 bytes word and then add that in array by load and increment and jump to the loop and return.

Code:

#Assembly code

.data

array: .word 20,30,40

.text

.globl main

main:

la a0, array

li a1, 3

jal ra, arraysum

li a7, 93

```
li a0, 0

ecall

.globl arraysum
arraysum:

    # a0 = int a[]

    # a1 = int size

    # t0 = ret

    # t1 = i

    li  t0, 0      # ret = 0

    li  t1, 0      # i = 0

loop:   # For loop

    bge t1, a1, end    # if i >= size, break

    slli t2, t1, 2    # Multiply i by 4 (1 << 2 = 4)

    add  t2, a0, t2    # Update memory address

    lw   t2, 0(t2)    # Dereference address to get integer

    add  t0, t0, t2    # Add integer value to ret

    addi t1, t1, 1    # Increment the iterator

    j    loop        # Jump back to start of loop (1 backwards)

end:

    mv   a0, t0    # Move t0 (ret) into a0

    ret    # Return via return address register
```

Output:

Run Step Prev Reset Dump Trace				tp (x4) 0
PC	Machine Code	Basic Code	Original Code	t0 (x5) 90
0x0	0x10000517	auipc x10 65536	la a0, array	t1 (x6) 3
0x4	0x00050513	addi x10 x10 0	la a0, array	t2 (x7) 40
0x8	0x00300593	addi x11 x0 3	li a1, 3	s0 (x8) 0
0xc	0x010000EF	jal x1 16	jal ra, arraysum	s1 (x9) 0
0x10	0x05D00893	addi x17 x0 93	li a7, 93	
0x14	0x00000513	addi x10 x0 0	li a0, 0	
0x18	0x00000073	ecall	ecall	
0x1c	0x00000293	addi x5 x0 0	li t0, 0 # ret = 0	

Task 5:

Description:

In this task, we were asked to generate the table of two by writing the integers in .data section and then in main by using the load word command, we first load address of data section inputs and then load the individual words by offset of 4 in register as a word takes 4 bytes in RISC-V and verify this by simulation also and in last for ecall we load immediate at a7 and at a0 with 0 and exit.

Code:

```
.data

table_of_two: .word 2, 4, 6, 8, 10, 12, 14, 16, 18, 20

.text

.globl main

main:

    la t0, table_of_two

    lw t1, 0(t0)

    lw t2, 4(t0)

    lw t3, 8(t0)
```

lw t4, 12(t0)

lw t5, 16(t0)

lw t6, 20(t0)

lw s0, 24(t0)

lw s1, 28(t0)

lw s2, 32(t0)

lw s3, 36(t0)

li a7, 93

li a0, 0

ecall

Output:

Run Step Prev Reset Dump Trace				a4 (x14)	0
0x10	0x0082AE03	lw x28 8(x5)	lw t3, 8(t0)	a5 (x15)	0
0x14	0x00C2AE83	lw x29 12(x5)	lw t4, 12(t0)	a6 (x16)	0
0x18	0x0102AF03	lw x30 16(x5)	lw t5, 16(t0)	a7 (x17)	0
0x1c	0x0142AF83	lw x31 20(x5)	lw t6, 20(t0)	s2 (x18)	18
0x20	0x0182A403	lw x8 24(x5)	lw s0, 24(t0)	s3 (x19)	20
0x24	0x01C2A483	lw x9 28(x5)	lw s1, 28(t0)		
0x28	0x0202A903	lw x18 32(x5)	lw s2, 32(t0)		
0x2c	0x0242A983	lw x19 36(x5)	lw s3, 36(t0)		
0x30	0x05D00893	addi x17 x0 93	li a7, 93		